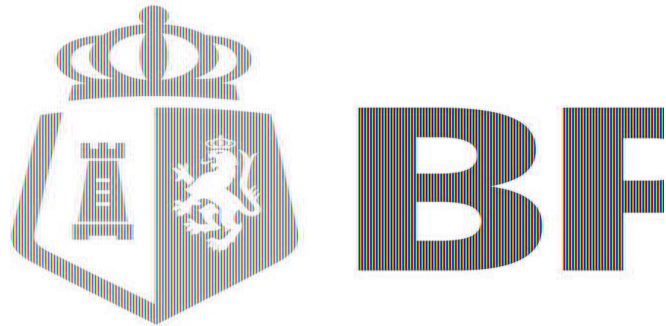




Java Exception Handling & Logging with SLF4J + Logback

A BPI Training Program

Welcome to Part 3 - Best Practices

- Best practices for error handling
- Common mistakes to avoid

Practice #1 - Be Specific with Exception Types

Code Demo - BAD Practice:

```
// BAD: Too generic
public void processUserData(String userId) {
    try {
        int id = Integer.parseInt(userId);
        User user = database.findUser(id);
        user.updateLastLogin();
    } catch (Exception e) { // ❌ Too broad!
        System.out.println("Something went wrong");
    }
}
```

Code Demo - GOOD Practice:

```
// GOOD: Specific exception handling
public void processUserData(String userId) {
    try {
        int id = Integer.parseInt(userId);
        User user = database.findUser(id);
        user.updateLastLogin();
    } catch (NumberFormatException e) {
        System.out.println("Invalid user ID format: " + userId);
    } catch (UserNotFoundException e) {
        System.out.println("User not found: " + userId);
    } catch (DatabaseException e) {
        System.out.println("Database error: " + e.getMessage());
    }
}
```

WHY: When you catch specific exceptions, you can provide appropriate responses for different error situations. It also makes your code more readable and maintainable.

WHEN: Always catch the most specific exception type that you expect might occur.

Practice #2 - Don't Swallow Exceptions

Code Demo - BAD Practice:

```
// BAD: Swallowing exceptions
public void saveDocument(String filename, String content) {
    try {
        FileWriter writer = new FileWriter(filename);
        writer.write(content);
        writer.close();
    } catch (IOException e) {
        // ✗ Empty catch block - exception is swallowed!
    }
    System.out.println("Document saved successfully");
}
```

Code Demo - GOOD Practice:

```
// GOOD: Proper exception handling
public void saveDocument(String filename, String content) {
    try {
        FileWriter writer = new FileWriter(filename);
        writer.write(content);
        writer.close();
        System.out.println("Document saved successfully");
    } catch (IOException e) {
        logger.error("Failed to save document: {}", filename, e);
        System.err.println("Error: Could not save document");
        // Could also re-throw or handle differently
    }
}
```

WHY: When you catch an exception and do nothing, you hide problems. The program continues running, but in a broken state. This makes bugs extremely hard to find.

WHEN TO AVOID: Never use empty catch blocks in production code.



Practice #3 - Preserve the Original Exception

Code Demo - BAD Practice:

```
// BAD: Losing the original exception
public void loadConfiguration(String configFile) throws ConfigurationException {
    try {
        Properties props = new Properties();
        FileInputStream input = new FileInputStream(configFile);
        props.load(input);
    } catch (IOException e) {
        // ✗ Original exception is lost!
        throw new ConfigurationException("Failed to load configuration");
    }
}
```

Code Demo - GOOD Practice:

```
// GOOD: Preserving the original exception
public void loadConfiguration(String configFile) throws ConfigurationException {
    try {
        Properties props = new Properties();
        FileInputStream input = new FileInputStream(configFile);
        props.load(input);
    } catch (IOException e) {
        // ✓ Original exception is preserved
        throw new ConfigurationException("Failed to load configuration: " + configFile, e);
    }
}
```

WHY: The original exception contains the root cause and the full stack trace. Without it, debugging becomes nearly impossible.

WHEN: Whenever you're wrapping an exception in a custom exception or a different exception type.



Practice #4 - Use Finally for Cleanup (or Try-with-Resources)

Code Demo - Traditional Finally:

```
// Traditional approach with finally
public String readFile(String filename) throws IOException {
    BufferedReader reader = null;
    try {
        reader = new BufferedReader(new FileReader(filename));
        return reader.readLine();
    } finally {
        if (reader != null) {
            try {
                reader.close(); // Always executes
            } catch (IOException e) {
                logger.warn("Failed to close reader", e);
            }
        }
    }
}
```

Code Demo - BETTER: Try-with-Resources:

```
// BETTER: Try-with-resources (Java 7+)
public String readFile(String filename) throws IOException {
    try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
        return reader.readLine();
        // reader is automatically closed, even if exception occurs
    }
}
```

WHY: Unclosed resources cause memory leaks, file locks, and database connection exhaustion.

WHEN: Whenever you work with resources that need cleanup



Practice #5 - Don't Use Exceptions for Control Flow

Code Demo - BAD Practice:

```
// BAD: Using exceptions for control flow
public Integer findUserAge(String userId) {
    try {
        return Integer.parseInt(userId);
    } catch (NumberFormatException e) {
        // ✗ Using exception to check if it's a number
        return null;
    }
}

// BAD: Using exceptions for business logic
public void processOrders() {
    try {
        while (true) {
            Order order = orderQueue.getNext();
            processOrder(order);
        }
    } catch (NoMoreOrdersException e) {
        // ✗ Using exception to exit loop
        System.out.println("All orders processed");
    }
}
```

Code Demo - GOOD Practice:

```
// GOOD: Validate before parsing
public Integer findUserAge(String userId) {
    if (userId == null || !userId.matches("\\d+")) {
        return null; // ✓ Normal validation
    }
    return Integer.parseInt(userId);
}

// GOOD: Check condition normally
public void processOrders() {
    while (orderQueue.hasNext()) { // ✓ Normal condition
        Order order = orderQueue.getNext();
        processOrder(order);
    }
    System.out.println("All orders processed");
}
```

WHY: Exceptions are expensive performance-wise. They also make code harder to read and understand.

WHEN TO AVOID: Don't use exceptions for predictable conditions or business logic.



Practice #6 - Provide Meaningful Exception Messages

Code Demo - BAD Practice:

```
// BAD: Vague messages
public void transferMoney(String fromAccount, String toAccount, double amount)
    throws InvalidTransferException {
    if (amount <= 0) {
        throw new InvalidTransferException("Invalid amount"); // ❌ Not helpful
    }
    if (fromAccount.equals(toAccount)) {
        throw new InvalidTransferException("Error"); // ❌ What error?
    }
}
```

Code Demo - GOOD Practice:

```
// GOOD: Descriptive messages with context
public void transferMoney(String fromAccount, String toAccount, double amount)
    throws InvalidTransferException {
    if (amount <= 0) {
        throw new InvalidTransferException(
            String.format("Transfer amount must be positive. Received: %.2f", amount)
        );
    }
    if (fromAccount.equals(toAccount)) {
        throw new InvalidTransferException(
            String.format("Cannot transfer from account %s to itself", fromAccount)
        );
    }
}
```

WHY: Good error messages save hours of debugging time. They should answer: What happened? Where? Why?

WHEN: Every time you throw an exception.

Practice #7 - Use Appropriate Log Levels

WHY: In production, you typically only keep ERROR and WARN logs. If you log everything as INFO, you'll drown in useless information.

WHEN: Follow the guidelines for each level."

Guidelines:

ERROR: System failures, critical problems that need immediate attention

WARN: Potential problems, deprecated features, recoverable errors

INFO: Important business events, startup/shutdown, major state changes

DEBUG: Detailed information useful for diagnosing problems

TRACE: Very detailed information, usually only in development

Code Demo:

```
public class OrderProcessor {
    private static final Logger logger = LoggerFactory.getLogger(OrderProcessor.class);

    public void processOrder(Order order) {
        logger.info("Processing order: {}", order.getId()); // ✅ Business event

        try {
            logger.debug("Order details: {}", order); // ✅ Detailed info for debugging

            validateOrder(order);
            logger.debug("Order validation passed");

            saveOrder(order);
            logger.info("Order {} completed successfully", order.getId()); // ✅ Important event

        } catch (ValidationException e) {
            logger.warn("Order {} failed validation: {}", order.getId(), e.getMessage()); // ⚠️ Recoverable problem
        } catch (DatabaseException e) {
            logger.error("Critical error processing order {}", order.getId(), e); // ❌ Critical failure
            throw e;
        }
    }
}
```



Practice #8 - Use Parameterized Logging

Code Demo - BAD Practice:

```
// BAD: String concatenation
logger.debug("User " + user.getName() + " logged in from " + user.getIpAddress()); // ✗

// BAD: String.format
logger.debug(String.format("Processing %d items for user %s", items.size(), userId)); // ✗
```

Code Demo - GOOD Practice:

```
// GOOD: Parameterized logging
logger.debug("User {} logged in from {}", user.getName(), user.getIpAddress()); // ✓

logger.debug("Processing {} items for user {}", items.size(), userId); // ✓

// Multiple parameters
logger.info("Transfer of ${} from account {} to {} completed", amount, fromAcct, toAcct); // ✓
```

WHY: Parameterized logging is more efficient and prevents security issues. The string is only built if that log level is enabled.



Practice #9 - Log the Exception Object

Code Demo - BAD Practice:

```
// BAD: Only logging the message
try {
    performOperation();
} catch (Exception e) {
    logger.error("Operation failed: " + e.getMessage()); // ✗ Stack trace lost!
}

// BAD: Converting exception to string
try {
    performOperation();
} catch (Exception e) {
    logger.error("Operation failed: {}", e.toString()); // ✗ Still no stack trace!
}
```

WHY: This includes the full stack trace in the logs, which is essential for debugging.

Code Demo - GOOD Practice:

```
// GOOD: Passing exception as last parameter
try {
    performOperation();
} catch (Exception e) {
    logger.error("Operation failed", e); // ✓ Full stack trace logged
}

// GOOD: With context and exception
try {
    processPayment(orderId);
} catch (Exception e) {
    logger.error("Payment processing failed for order {}", orderId, e); // ✓ Context + stack trace
}
```



Practice #10 - Don't Log Sensitive Information

Code Demo - BAD Practice:

```
// BAD: Logging sensitive data
public void authenticateUser(String username, String password) {
    logger.debug("Login attempt - Username: {}, Password: {}", username, password); // ✗ NEVER!
}

public void processPayment(CreditCard card) {
    logger.info("Processing payment with card {}", card.getNumber()); // ✗ Dangerous!
}
```

Code Demo - GOOD Practice:

```
// GOOD: Masking sensitive data
public void authenticateUser(String username, String password) {
    logger.debug("Login attempt - Username: {}", username); // ✔ No password!
}

public void processPayment(CreditCard card) {
    String maskedCard = "****-****-****-" + card.getLastFourDigits();
    logger.info("Processing payment with card {}", maskedCard); // ✔ Masked
}

// GOOD: Helper method for masking
private String maskEmail(String email) {
    if (email == null || !email.contains("@")) return "invalid";
    String[] parts = email.split("@");
    return parts[0].charAt(0) + "****@" + parts[1]; // j***@example.com
}
```

WHY: Logs are often stored in plain text and may be accessible to many people. Logging sensitive data creates security vulnerabilities.

WHEN TO AVOID: Always sanitize sensitive information before logging.



Practice #11 - Use Logger Per Class

Code Demo - GOOD Practice:

```
public class UserService {
    //  Logger named after the class
    private static final Logger logger = LoggerFactory.getLogger(UserService.class);

    public void createUser(User user) {
        logger.info("Creating new user: {}", user.getUsername());
        // ...
    }
}

public class PaymentService {
    //  Each class has its own logger
    private static final Logger logger = LoggerFactory.getLogger(PaymentService.class);

    public void processPayment(Payment payment) {
        logger.info("Processing payment: {}", payment.getId());
        // ...
    }
}
```

WHY: This allows you to control logging per package or class and makes it easy to identify where log messages come from.

Configuration Example (logback.xml):

```
<!-- You can now control logging per class/package -->
<logger name="com.company.UserService" level="DEBUG"/>
<logger name="com.company.PaymentService"
level="INFO"/>
<logger name="com.company.database" level="WARN"/>
```

SEATWORK #9: Your Turn to Try! (Branch: M4_Activity9)

Time: 15-20 minutes

Your Task: Complete logging configuration and implement exception handling

Instructions:

1. **Create logback.xml in src/:**
 - Console appender (pattern with timestamp, level, logger, message)
 - Daily Rolling File appender:
 - Current file: `logs/bank-app.log`
 - Archive pattern: `logs/bank-app-%d{yyyy-MM-dd}.log`
 - Keep 30 days of logs
 - Total size cap: 1GB
 - Root logger level: INFO
2. **Complete BankAccountService.java (30 TODOs):**
 - Add logger statements (INFO, WARN, ERROR, DEBUG)
 - Use parameterized logging with `{ }`
 - Throw exceptions with meaningful messages
 - Follow the parameter hints in each TODO
3. **Run BankAccountServiceTest.java** to verify your work
4. **Check results:**
 - Console output for test results
 - `logs/bank-app.log` file for logged messages

Files Provided:

- `BankAccountService.java` (30 TODOs to complete)
- `BankAccountServiceTest.java` (ready to run)
- Exception classes (complete)
- `BankAccount.java` (complete)

Summary

What We Learned (Module 4 - Complete):

Part 1: Exception Handling

- Checked vs Unchecked exceptions
- try-catch-finally blocks
- throw vs throws
- Custom exceptions

Part 2: Logging with SLF4J + Logback

- Log levels (ERROR, WARN, INFO, DEBUG)
- Logback configuration
- Parameterized logging

Part 3: Best Practices

- Be specific with exceptions
- Never swallow exceptions
- Use appropriate log levels
- Log exception objects
- Avoid common mistakes

Resources & References

Java Exception Handling:

- <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- <https://www.baeldung.com/java-exceptions>
- <https://www.geeksforgeeks.org/exceptions-in-java/>
- <https://www.javatpoint.com/exception-handling-in-java>

SLF4J:

- <https://www.slf4j.org/>

Logback:

- <https://logback.qos.ch/>

Thank You!

Congratulations on Completing Module 4!
Exception Handling & Logging with SLF4J + Logback

You Now Know How To:

- Handle errors professionally
- Create meaningful error messages
- Configure logging frameworks
- Write maintainable, debuggable code
- Avoid common pitfalls

Remember:

"The best error message is the one that helps you fix the problem at 2 AM"

M4_PROJECT:

Your Task: Implement exception handling and configure logging

Instructions:

- 1. Create a new branch of the latest M3 Project and name it M4GroupProject1.**
- 2. Configure logging and Implement exception handling.**